Systems Administration (ASIST)

Practical Theoretical Class 08

Windows – Linux integration
Samba software
Event Logs

Linux – Windows integration

Interoperability between Windows and Linux servers across the network is now facilitated by the fact that both increasingly integrate standard protocols such as DNS, LDAP, iSCSI, NFS, SMB / CIFS and Kerberos 5, among others.

Despite this support for basic application protocols, the way they are used differs substantially between Windows servers and Linux servers, essentially due to design differences between the underlying operating systems.

First of all, most *Active Directory* domain concepts from Windows servers are missing from Linux servers.

Samba (*https://www.samba.org/*) is a set of software components for Linux that includes network services and client applications. It allows a Linux server to simulate many of the features of Windows servers and is the base tool for any more serious integration between these two types of networked systems.

User and group accounts

While on Linux users and groups are identified by a unique **UID** and **GID** respectively, on Windows they are identified by a unique *Security Identifier* (**SID**).

Having this difference in mind, matching will require the establishment of a mapping mechanism (***UID <-> SID***) and (***GID <-> SID***).

In addition to this, the incompatibilities are diverse starting with usernames and groups that are much more restrictive in Linux than in Windows.

One way to resolve name mismatches is to create static mappings between Windows names that are not valid on Linux and names that are valid on Linux. For example, matching the Windows **Domain Users** group to a group created on Linux **dom_users**.

Note also that most restrictions on usernames and groups on Linux do not actually exist, they are backward compatibility restrictions, generally the names supported on Windows are directly supported on Linux - except for the use of spaces in the name.

Both on Linux and Windows each user must belong to a primary group (POSIX compatibility) and optionally to other groups. However, while on Linux only users can be members of groups, in Active Directory groups can be members of groups.

File sharing

From an iSCSI SAN standpoint, there are never integration issues because logical disks are formatted and used by the operating system that receives them as local disks.

At Network File Sharing (NAS) level, Windows servers are SMB/CIFS-oriented while Linux servers are NFS-oriented.

Although Windows servers can operate as NFS clients and servers, the best integration solution is to add SMB / CIFS support to Linux servers through Samba.

Samba allows you to share folders and printers via SMB / CIFS fully compatible with Windows machines, incompatibilities arise from the point of view of the file system characteristics underlying the sharing.

In Linux the base ACLs (Posix) only supports three entities: **owner**, **group** and **others**, each one with **read**, **write** and **execute** permissions.

As long as the object-associated UID (*owner*) corresponds to a Windows user (*SID*) and the object-associated GID (*group*) matches a Windows group (*SID*), these two entities are directly placed in the Windows ACL. The entity Linux **others** is normally associated with the Windows group **Everyone**.

Linux - *Extended Permissions* (ACL)

On current Linux systems the base ACLs can be expanded using the **getfacl/setfacl** commands. Only the three base ACL (*rwx*) permissions still exist, but entities are no longer limited to (*owner/group/others*). In addition to these three basic FTA entities other users and groups may be added.

Using the **setfacl** command it is possible to set a folder to "*Default ACL*", this ACL will be inherited by objects placed inside that folder. You can also define a maximum effective permissions mask that can be assigned by various ACL entries.

Both Samba (SMB / CIFS) and NFS network shares support Linux's extended permissions, while Windows ACLs allow better integration through Samba.

On a network share made via Samba the defined ACLs are stored as *extended permissions* on Linux, so they are no longer limited to only three entities.

Note that if Samba is acting as an AD domain DC the share ACLs will be native Windows ACLs and are not stored as *extended permissions* on Linux. For this reason it is advised that DC Samba *Active Directory* does not perform the **File Server** role and that role is transferred to a *domain member*.

Samba – MS-DOS attributes

Some applications still use the attributes of legacy MS-DOS files: ***System***, ***Hidden***, ***Archive*** and ***Read-only***. These attributes do not exist on Linux file systems underlying network shares performed through Samba.

The problem can be addressed by storing it in the Linux *extended permissions* ACL or by mapping it to execute permissions from **owner**, **group** and **others**.

Samba – printer sharing

Printer sharing (Print Server) is fully supported by Samba, including automatic *driver* installation on Windows client operating systems.

Samba can provide locally configured printers through *Common UNIX Printing System* (**CUPS**) or **LPRng**, or directly use network printers.

It is also possible to create a virtual PDF printer that automatically creates PDF files in a shared folder on the network.

Linux/Windows integration - Samba

Since version 4 Samba can assume the *Domain Controller Active Directory* role, which means it can do just about everything Windows Server does as DC in *Active Directory*. It is even possible to have the same domain *Domain Controllers* Windows Server 2016 working in conjunction with *Domain Controllers* Samba.

This will be the most advanced role that Samba supports, however there are many integration scenarios in which it can be used.

Samba – *Standalone Server*

Samba configured as ***Standalone Server*** allows network shares to be made available. Not being integrated in any domain, user authentication will be required before the server to access the shares.

Valid user accounts will be all valid Linux accounts (**NSS**) as long as they have been added to the Samba Windows account database.

Samba must maintain a shadow database of Linux user and group accounts to store attributes that, when required on Windows accounts, do not exist on Linux accounts.

In this type of configuration (***Standalone Server***) Samba must simulate Windows user and group accounts from Linux accounts.

The only attribute of Linux accounts that is directly transferred to Windows accounts is **login-name,** everything else has to be set by Samba and stored in a shadow repository. User and group SIDs are defined based on their UID and GID, respectively, and are stored together with all other Windows account attributes.

Windows performs user authentication through ***hashes*** called **NT-password** and **LM-password** (the latter is only kept for backward compatibility with older clients). These ***hashes*** are also stored in Windows user accounts.

The Linux password and the Windows password are totally independent, however Samba includes a mechanism whereby whenever the user changes the password through Windows the respective Linux password is changed in the same way.

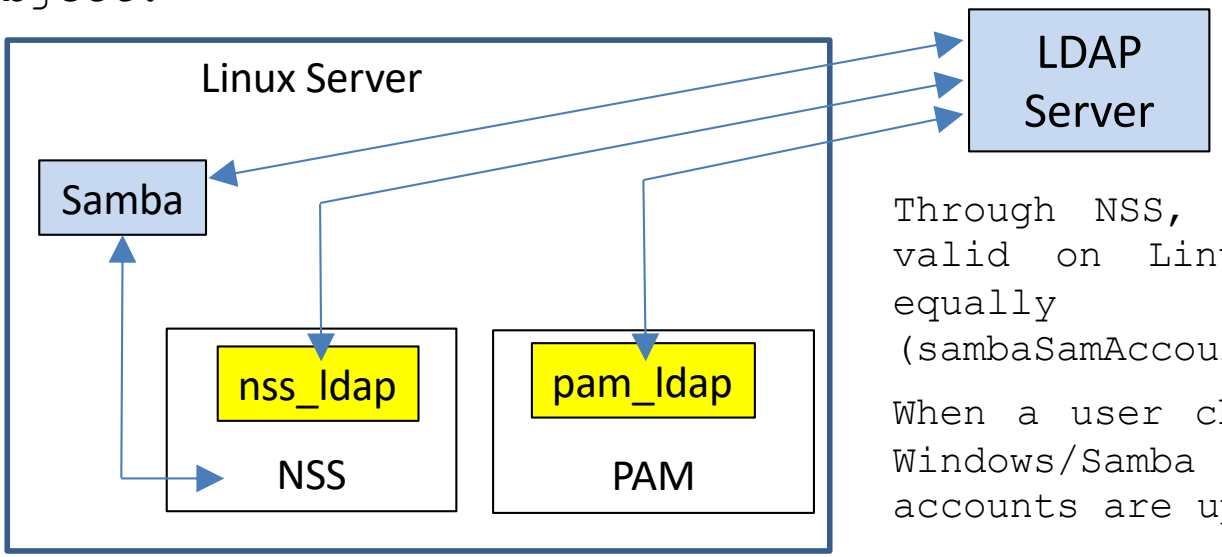
A user or group account is only valid from a Samba/Windows perspective if it is a valid Linux user/group and exists in the Samba shadow repository (Windows accounts).

Samba – Using LDAP as a repository

In any Samba configuration where you need to manage local Windows user accounts, Samba defaults to storing that data in internal databases (**tdbsam** – *Trivial Data Base Security Account Manager*). In this case we have stored in different places the Linux (*NSS*) account and the Windows shadow account (*tdbsam*), using LDAP allows to solve this problem with some advantages.

In LDAP we can define objects that belong to several classes (provided only one is structural and the other auxiliaries).

For example, in the case of user accounts the **posixAccount** and **sambaSamAccount** LDAP classes can be used. This allows storing Linux (*posixAccount*) and Windows (*sambaSamAccount*) attributes on the same object.



Through NSS, users defined in LDAP are valid on Linux (posixAccount) and are equally valid for Samba (sambaSamAccount).

When a user changes his password through Windows/Samba the passwords of both accounts are updated.

Samba – *Active Directory Domain Member*

Samba can be a ***Domain Member Server*** from an *Active Directory* domain or from a Windows NT domain, regardless of whether the domain operates with Windows Server or Samba *Domain Controllers*.

This means that all domain objects, including users and groups, will become valid on the Linux system and hence on Samba. This will allow authenticated users in a forest domain to have direct access to network shares provided by Samba without additional authentication.

Adapting user and group accounts as defined in the Windows domain to what Linux expects to be user and group accounts takes some effort. There is a local service dedicated to this task called **Winbind.**

Winbind is responsible for giving a Linux look to Windows accounts and has several challenges ahead. The most important thing is to map the SIDs of Windows users and groups to UID and GID on Linux.

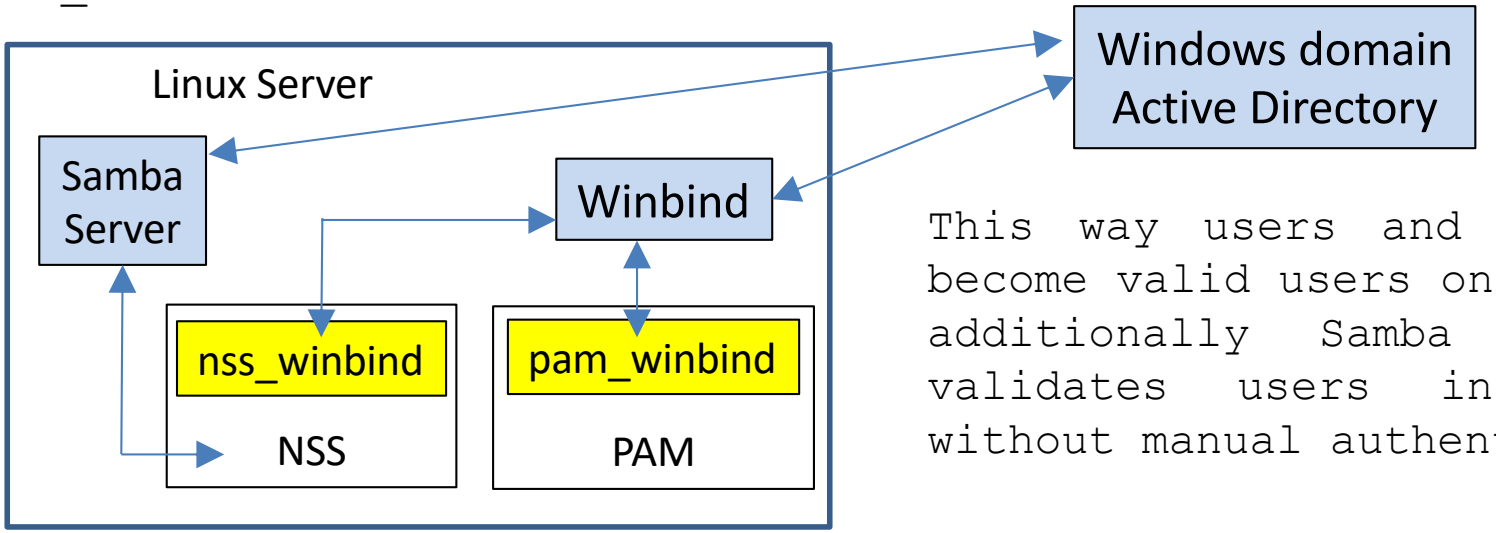
In this type of configuration, a range of UIDs and GIDs is assigned for Winbind to handle, each time it encounters an unknown SID it is persistently associated with a new UID or GID of the established ranges, depending on whether it is a user or a group.

Samba *domain member – Winbind*

Winbind keeps mappings established in an internal database, so when assigning a UID or GID to a particular SID, the corresponding user or group will always have the same UID or GID.

However, if there are multiple *domain members* configured in this way, there is no guarantee that in all of them Winbind will assign the same UID or GID to the same SID. To solve this problem it is possible to configure Winbind to store mappings in a centralized LDAP repository, thus ensuring that a Windows user or group will have the same UID or GID on all Linux servers that are members of the domain.

Winbind users and groups are integrated into the Linux system at the expense of appropriate NSS and PAM modules, namely nss_winbind and pam_winbind:

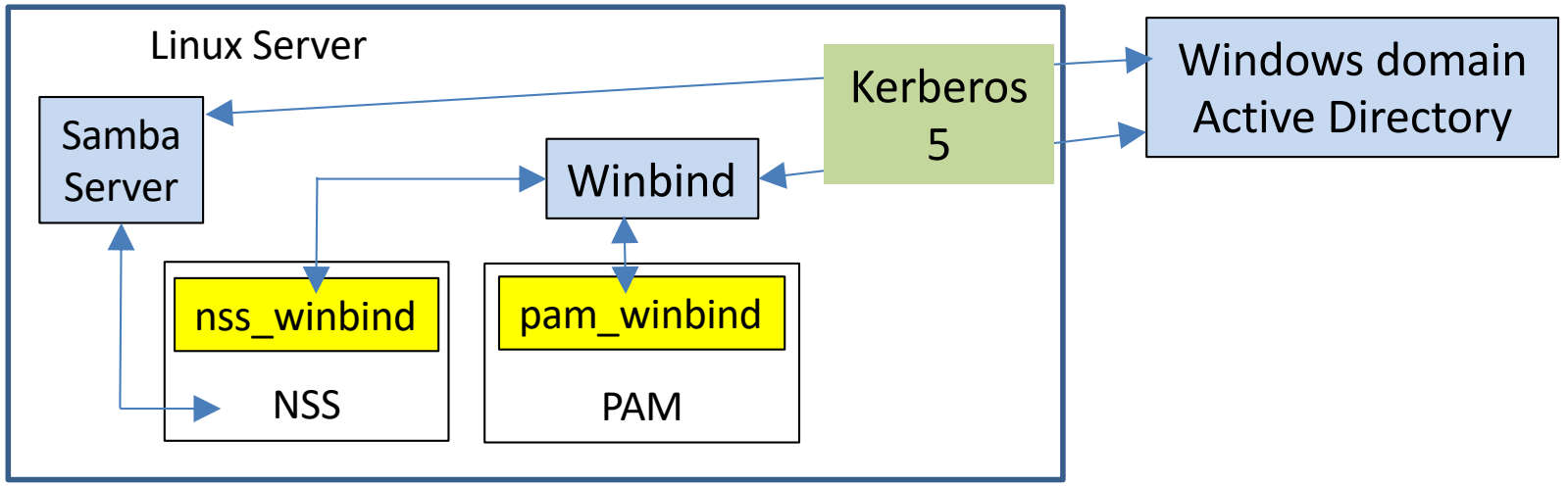


This way users and domain groups become valid users on Linux system, additionally Samba automatically validates users in the domain without manual authentication.

Samba *domain member – Kerberos 5*

Active Directory operation requires that all transactions between domain members be protected and authenticated through *Kerberos 5*, *Domain Controllers* have a *Kerberos 5* server, including a *Key Distribution Center* (**KDC**).

So ***domain members in an Active Directory domain*** need to have a Kerberos 5 client configured to use this KDC.



To prevent ***replay*** attacks messages used by Kerberos carry ***timestamps***, usually a message becomes invalid if the time difference is over 300 seconds.

It is therefore essential to ensure time synchronization between the machines in an *Active Directory* domain. The best way to ensure this is to use the *Network Time Protocol* (**NTP**) service.

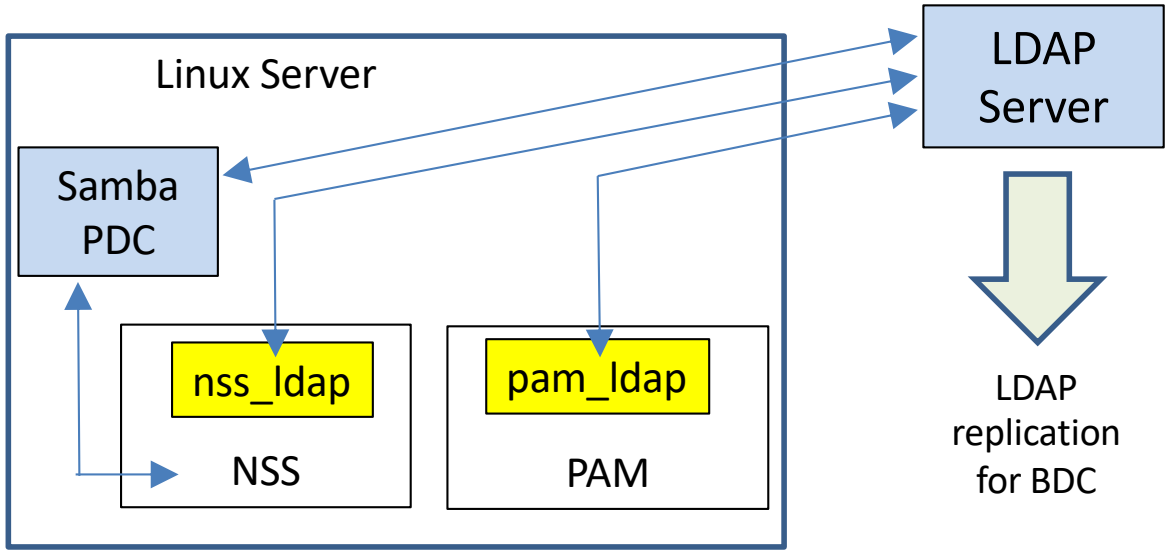
Samba *domain controller NT*

Samba can function as a **PDC** (*Primary Domain Controller*) for a Windows domain in NT compatibility mode (*pre Active Directory*).

The configuration is similar to the ***Standalone Server*** operating mode, ie local Linux accounts backed up by a database with supplemental Windows attribute information are used.

The most interesting solution is to use an LDAP database to store account information in an integrated manner.

All features present in native NT domains are supported, namely: Domain Single Sign-On, home directory, roaming profiles, logon script and domain policies.



Using LDAP, it is also simple to define BDCs (*Backup Domain Controllers*), for this purpose LDAP replication mechanisms should be used to ensure the replication of the PDC LDAP database to those used by the various BDCs.

Samba *Active Directory Domain Controller*

Samba can also operate as *Domain Controller Active Directory*, this currently requires some conditions on Linux server configuration.

Due to the specifics of *Active Directory*, a Samba internal LDAP server must be used and a Samba integrated DNS server must also be used. You can use the operating system Kerberos MIT server or alternatively an internal Samba Kerberos server.

Using integrated servers in Samba means that all user, group and DNS management will have to be done through Samba/Windows.

The integrated LDAP server is purely *Active Directory*, does not contain Linux account attributes. For example, for domain users and groups to be valid on Linux, you must use Winbind and configure Linux as a *domain member* as if it were a native Windows *Active Directory* domain. It is not possible to use the integrated LDAP server directly.

In an *Active Directory* domain, there must be at least two DCs, in this mode Samba ensures automatic replication of the *Active Directory* database between all Domain Controllers in the domain.

Event Logs

Operating systems and applications produce messages representative of events that occur, those messages are persistently stored in log files.

Analysis of log files by administrators should be the first step when something does not work as intended. Inspection of log files also makes it possible to detect anomalous situations which, while not affecting the apparent functioning of the systems, can be important, for example from a security point of view.

If automatic system monitoring tools are used, one of the aspects that should be monitored is the log files.

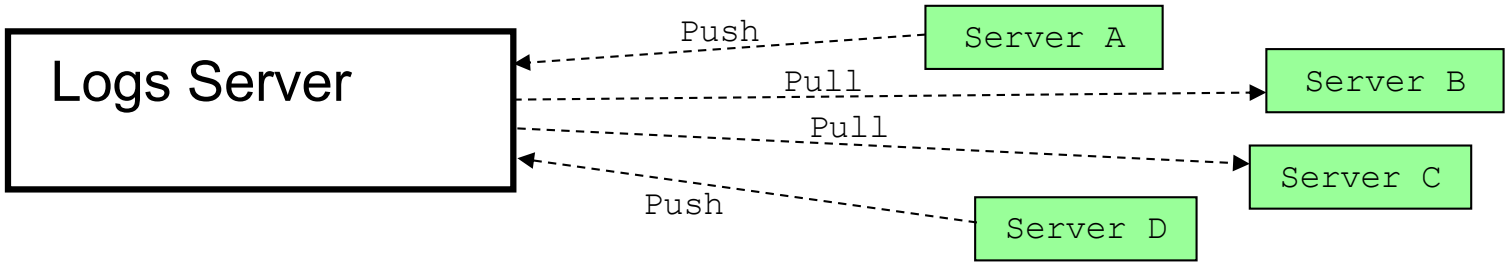
Each log message generated by an application, service, or operating system typically contains a **severity level**, a **source** identification (who produced it - computer, application, service, subsystem, etc.) and event **descriptive text**.

The amount of log information generated by each source can usually be configured on that source, typically a service or application. If a service or application is not working properly we can often configure it to increase the amount of logging information it generates (logging level). Once the problem is detected and resolved, we should again reduce the logging level to normal so as not to affect your performance.

Centralized Event Logs

In a distributed network environment where there are multiple servers and other equipment, we can centralize event logging. For this purpose will be chosen a server (log server) that will get through the network the events produced in all other equipment. This way the administrator has a unique place where he can consult the events related to all infrastructure equipment.

Log centralization can be implemented in either *Source Computer Initiated* (**PUSH**) or *Collector Initiated* (**PULL**) forms.



PULL: The log server takes the initiative to establish a session for log transfer, for this it will be necessary to define in the log server one by one all equipment from which events are to be obtained.

PUSH: Event source equipment takes the initiative to contact the log server. In each of the equipment it is necessary to define the same correct server logs.

Windows Event Logs

On Windows systems, events are stored in separate files according to their source. Event Viewer lets you view and manage these event logs.

The logs are organized by theme according to their origin (system, applications, services, etc.). There is a distinct log for each, in these logs all events are recorded regardless of their severity. New log files can also be added.

In **Event Viewer** we can create ***Custom Views*** of logs, in a *Custom View* we can aggregate sets of events from various logs, define the severity levels we want to include in *Custom View* and other filtering criteria.

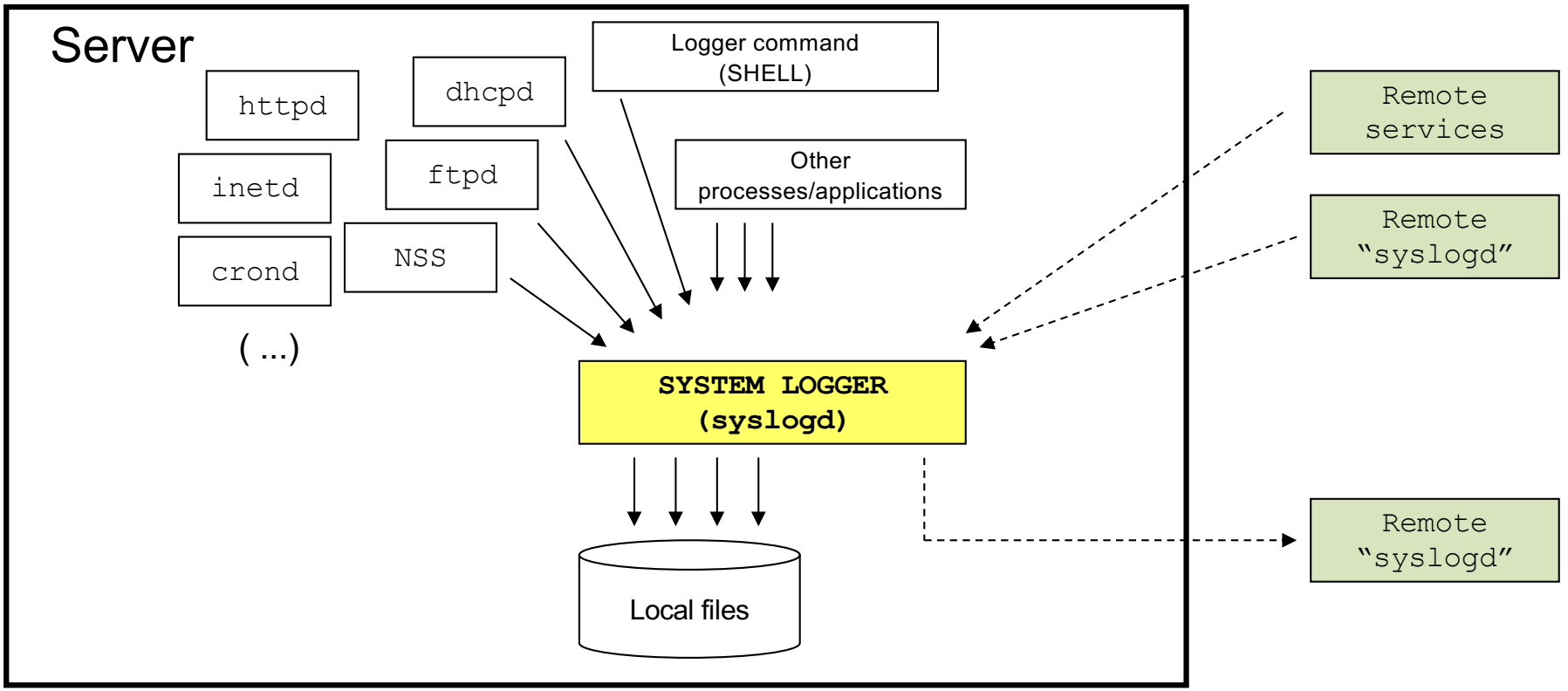
We can centralize the logs by creating a ***Subscription*** where we define the machines from which we want to get the events, the log where to deposit these events, and filter criteria.

In a *Subscription* we can get the events at the initiative of the server that wants to collect them (*Collector Initiated*) or at the initiative of the source machines (*Source Computer Initiated*). In the latter case the source machine must be configured for this purpose. Log transfer between machines uses HTTP/HTTPS protocol directed to special port numbers.

Linux - *System Logger* (syslog)

Linux uses a *standard* logging system known as **Syslog**, through an appropriate API, applications and services send their events to Syslog.

According to the attributes defined in the source, Syslog classifies events and, through the administrator defined settings, performs one or several actions on them. Usually one of the most common actions is appending the event to a text file that is used as a repository, usually in the **/var/log/** folder.



System Logger – Message classification

Although more advanced versions such as **RSYSLOG** (*the rocket-fast system for log processing*) allow you to use a wide variety of criteria to classify events, including through the content of their text, the most traditional form of classification is through ***facility*** and ***severity*** defined at source.

The ***facility*** attribute is for identifying the source type, can have one of the following values: *auth, authpriv, cron, daemon, kern, lpr, mail, mark, news, security* (the same as *auth*), *syslog, user, uucp* and *local0* to *local7*.

The ***severity*** attribute defines the event severity level, in ascending order of severity, possible values are: debug, info, notice, warning, warn (the same as previous), err, error (the same as previous), crit, alert, emerg, panic (the same as previous).

These two attributes are often presented in grouped form:
facility.severity

The fundamental mission of Syslog is to analyze these two attributes and, depending on their value, define what to do with the event. Some standard actions are: append to a local file; send to a remote syslog server; send directly to active user terminals; run na external program and pass .it the message ***stdin***.

Linux System Logger – Configuration

In traditional versions of Syslog the configuration file is **/etc/syslog.conf** and event classification is based solely on ***facility.severity*** attributes, the file contains a pattern sequence of these attributes and the action to be taken for events that. For each event, all actions set to matching patterns will be performed, not just the first match.

Example:

```
kern.*                          /var/adm/kernel
mail.*;mail.!=info              -/var/adm/mail
*.alert                         root,admin
mail.=info                      @server2.ipp.pt
mail,news.=info                 *
kern.info;kern.!err             |/root/kern-info
```

Severity values in the pattern match events with severity level greater than or equal to. For each pattern an action is specified, if the action starts with **/** means that this is a log file where the event is to be logged, it may be preceded by the signal **-** to indicate it is necessary to run **sync** (otherwise it may not be immediately written to the file).

```
*.alert                         root,admin
mail.=info                      @server2.ipp.pt
mail,news.=info                 *
kern.info;kern.!err             |/root/kern-info
```

The action can also be one or a user list, in which case if users are active in terminal sessions they receive a message with the event on their terminal. The ***** symbol allows sending this message to all active users in terminal sessions. The pipe (**|**) identifies an external command to execute that receives through ***stdin*** the event.

The **@** symbol allows us to identify, by means of an IP address or DNS name, a machine to which we want to send the event log. A Syslog configured to accept network events at UDP port number 514 must be running on the remote machine.

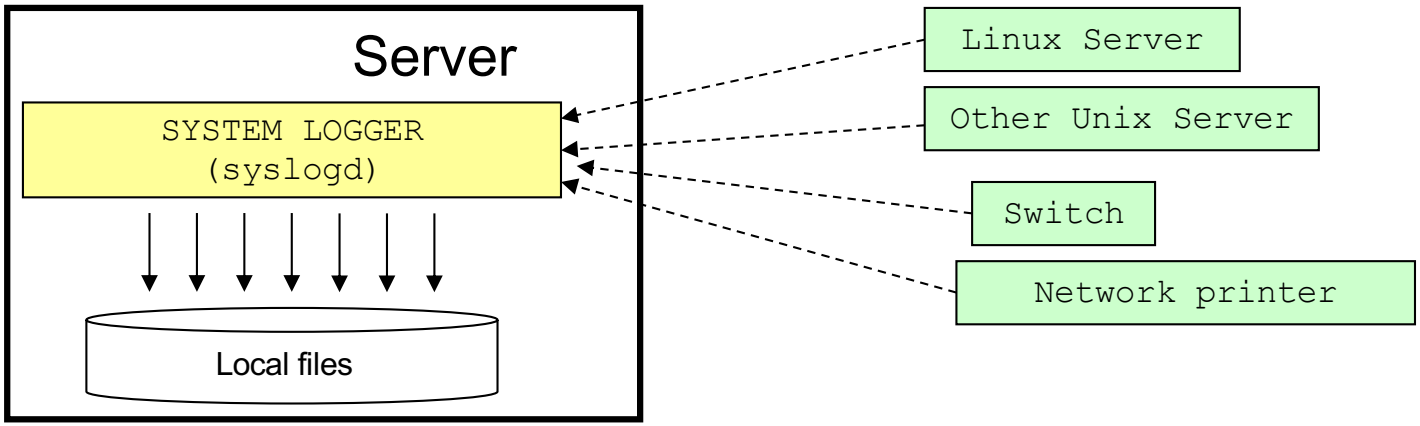
Some versions of Syslog also support the use of TCP and other port numbers, in which case the action must be identified by **@@** and the port number can be set, by default it will be port 514.

Example:

```
*.*             @@server1.example.net:10514
*.info          @192.168.5.7:23000
*.!=info        @@192.168.5.1
```

Linux System Logger – Centralization

An advantage of Syslog is that it is a standard (RFC5424), which among other things defines as minimum requirements the transport of events through UDP. A wide range of simple network devices such as printers, switches, routers and more support the Syslog protocol.



It is even possible through DHCP to provide network equipment with an indication of which Syslog server address to use (DHCP **log-servers** attribute).

The event transfer protocol over the Syslog network has no authentication and security mechanisms, so it should be used over secure networks, preferably using TCP and allowing access only to the source addresses corresponding to the equipment for which it is intended to receive events (filtering).

Although the Syslog protocol is not supported by Microsoft on Windows systems, non-Microsoft implementations are available, including a version of **RSYSLOG.**

Linux System Logger – log files

Ultimately, most of the events processed by Syslog end up in log files. These are text files and as such can be consulted using various commands available in Linux for this purpose such as: **cat**, **less**, **tail**, **more**, **grep** and **cut**.

Logrotate

The size of each log file will grow successively, to organize these files you can use ***logrotate*** command.

Typically ***logrotate*** is run daily by CRON to consolidate log files according to the settings defined in the **/etc/logrotate.conf** file. Consolidation is a rotation of the log file, ie transferring the current content to an archive file and emptying the log file, the archive files have the same log name with a numeric order or date suffix.

Through this consolidation we can define for each log file: the periodicity of rotation, whether archive files should be compressed, and the maximum number of revolutions from which to begin deleting older archive copies.

Instead of a periodic rotation we can also define that the rotation will be performed when the log size exceeds a given limit. An email address can also be configured to which file copies exceeding the maximum number of rotations will be sent before being deleted.

Linux – registration of inputs and outputs

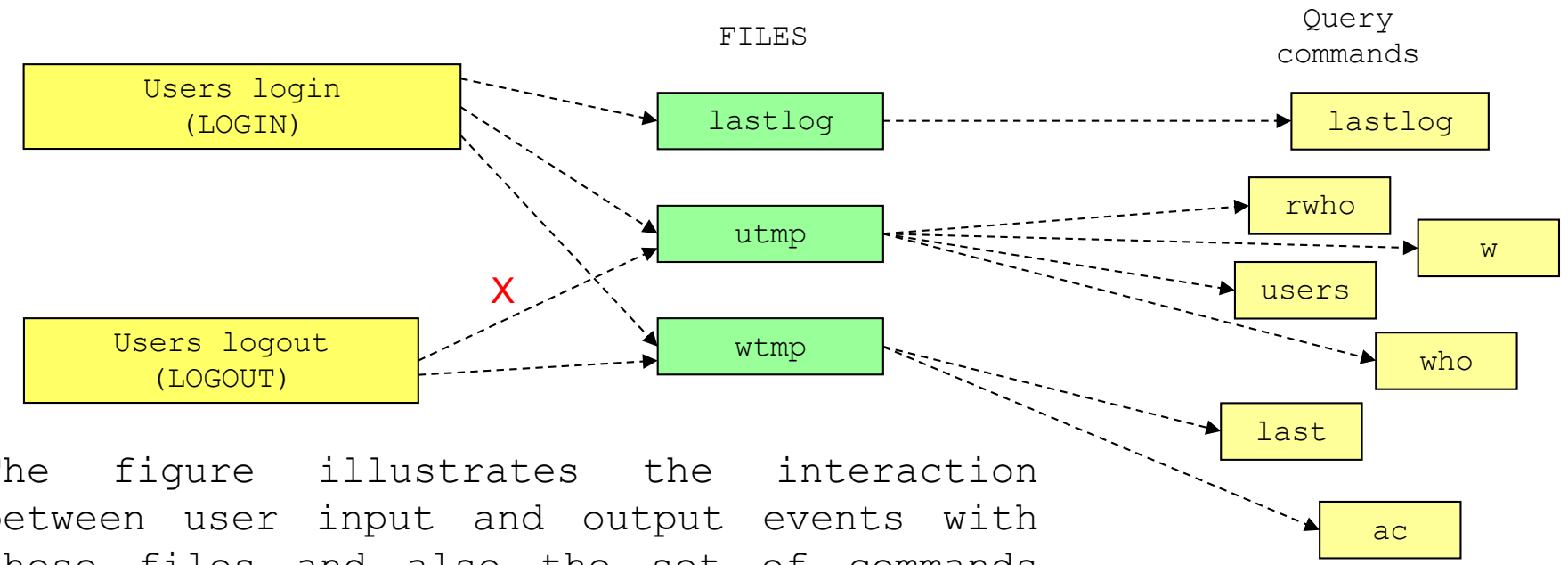
Although user login (logout) and logout (logout) events in the system are logged as events through Syslog (including failed access attempts), there are additional logs in Unix for this purpose. Programs that handle these events have a responsibility to log them through a separate Syslog API. These records are based on three files that are not in readable text format:

•**utmp** (usually **/var/run/utmp**) – contains, for each terminal, the terminal name, current user name, user source (remote machine/address) and login date/time. When the user logs out, the record is deleted.

•**lastlog** (usually **/var/log/lastlog**) – contains, for each UID, the time and place (terminal/remote machine/address) of the last system entry (login).

• **wtmp** (usually **/var/log/wtmp**) – contains the historical record of all inputs and outputs in the system.

Linux – registration of inputs and outputs



The figure illustrates the interaction between user input and output events with these files and also the set of commands used to query them. Example of result of **last** command:

```
[root@server ~]# last -5
andre    pts/0        beavis.dei.isep. Wed Nov 12 16:35   still logged in
dei      pts/0        beavis.dei.isep. Wed Nov 12 11:46 - 14:12  (02:25)
i090763  pts/4        srv3.dei.isep.ip Wed Nov 12 09:36 - 09:54  (00:17)
i080751  pts/3        srv3.dei.isep.ip Wed Nov 12 08:52 - 10:02  (01:10)
i040478  pts/2        srv2.dei.isep.ip Wed Nov 12 08:51 - 10:03  (01:11)

wtmp begins Sat Nov  1 12:59:40 2004
```